

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Выполнил:

Малахов Алексей

БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service.
2. Спроектировать микросервисную архитектуру: отразить взаимосвязь между микросервисами, описать способы взаимодействия.
3. Спроектировать разделение базы данных на обособленные БД.
4. Спроектировать и описать эндпоинты для межсервисного взаимодействия в формате спецификации OpenAPI.
5. Описать варианты запросов и ответов, задокументировать возможные ошибки.
6. Сформировать документ с конкретными требованиями и шагами для декомпозиции монолита.

Ход работы

Исходный монолит

Приложение является сервисом аренды недвижимости, реализованным на TypeScript + Express.js (routing-controllers, TypeORM, PostgreSQL). Монолит содержит 9 контроллеров и 13 таблиц, сгруппированных в 5 доменов:

- **Аутентификация:** регистрация, вход, JWT-токены.
- **Пользователи:** профиль, роли (арендодатель / арендатор).
- **Недвижимость:** объекты, фотографии, история цен, избранное.
- **Аренда и платежи:** договоры аренды, транзакции.
- **Коммуникация и отзывы:** диалоги, сообщения, отзывы об арендодателях.

Выделение микросервисов

Декомпозиция выполнена по принципу «один сервис — один бизнес-домен». Выделено 6 микросервисов:

Сервис	Ответственность	Контроллеры монолита
Auth Service	Регистрация, вход, выдача и обновление JWT-токенов, валидация токенов	AuthController
User Service	Профили пользователей, управление ролями, информация об арендодателях	ProfileController, LandlordController
Property Service	CRUD объектов недвижимости, фотографии, история цен, избранное, поиск	PropertyController, FavoriteController
Rental Service	Заявки на аренду, подтверждение, статусы, финансовые транзакции	RentalController, TransactionController
Messaging Service	Диалоги между пользователями, отправка и чтение сообщений	ConversationController

Сервис	Ответственность	Контроллеры монолита
Review Service	Отзывы арендаторов об арендодателях, рейтинг	ReviewController

Дополнительно вводится **API Gateway** — единая точка входа для всех внешних запросов. Gateway выполняет аутентификацию (проксирует JWT-токен в Auth Service), маршрутизацию и агрегацию ответов (BFF-паттерн).

Схема взаимодействия микросервисов

Все микросервисы взаимодействуют через синхронные REST-вызовы по внутренней сети. Внешние запросы проходят только через API Gateway. Все межсервисные вызовы идут по маршрутам вида `/internal/...`, недоступным извне.

От	К	Причина
API Gateway	Auth Service	Валидация JWT-токена при каждом входящем запросе
Auth Service	User Service	Создать профиль пользователя при регистрации
Property Service	User Service	Получить профиль арендодателя для карточки объекта
Rental Service	Property Service	Получить детали объекта при создании заявки на аренду
Rental Service	User Service	Получить профиль арендатора
Rental Service	Property Service	Обновить статус объекта при активации/завершении аренды
Review Service	Rental Service	Проверить наличие завершённой аренды перед созданием отзыва
Review Service	User Service	Получить профиль арендодателя
Messaging Service	User Service	Получить профили участников диалога

Разделение базы данных

Каждый микросервис владеет собственной базой данных PostgreSQL. Прямые JOIN между БД разных сервисов исключены; данные агрегируются на уровне сервиса через межсервисные вызовы или хранятся в денормализованном виде (кэш ID).

Сервис / БД	Таблицы
Auth DB	auth_users (id, email, password_hash, is_active, is_verified, created_at, updated_at, deleted_at)
User DB	users (id, first_name, last_name, phone, avatar_url, created_at, updated_at, deleted_at), user_roles (id, user_id, role, assigned_at)

Сервис / БД	Таблицы
Property DB	properties, property_photos, property_price_history, favorites
Rental DB	rentals, transactions
Messaging DB	conversations, conversation_participants, messages
Review DB	landlord_reviews

Таблица `users` монолита делится между Auth DB и User DB: Auth DB хранит только учётные данные (email, пароль, флаги активности), User DB — профильные данные. Первичный ключ — целочисленный `id` (auto-increment), генерируемый в Auth Service и передаваемый в User Service при создании пользователя.

Межсервисные эндпоинты (OpenAPI 3.0.3)

Спецификации **внутренних** эндпоинтов (/internal/...) оформлены в формате OpenAPI 3.0.3 и вынесены в отдельные файлы:

- openapi/auth-service.yaml
- openapi/user-service.yaml
- openapi/property-service.yaml
- openapi/rental-service.yaml

Эндпоинты недоступны извне — только через внутреннюю сеть. Во всех спецификациях аутентификация выполняется через заголовок X-Service-Token (схема apiKey). Формат ошибки единообразен:

```
{ "code": "NOT_FOUND", "message": "User with id 42 not found" }
```

Сводная таблица всех внутренних эндпоинтов:

Метод	Маршрут	Назначение
POST	/internal/auth/validate	Валидация JWT-токена (API Gateway → Auth Service)
POST	/internal/users	Создать профиль пользователя (Auth Service → User Service)
GET	/internal/users/{id}	Получить профиль пользователя по ID
GET	/internal/users?ids=...	Пакетное получение профилей (Messaging Service)
GET	/internal/properties/{id}	Получить данные объекта недвижимости
PATCH	/internal/properties/{id}/status	Обновить статус объекта (Rental Service)
GET	/internal/rentals/{id}	Получить данные аренды
GET	/internal/rentals/check	Проверить наличие завершённой аренды (Review Service)

Коды ответов, схемы запросов/ответов и примеры задокументированы в соответствующих YAML-файлах.

Общие требования к межсервисным запросам

Все внутренние запросы должны содержать заголовок X-Service-Token для аутентификации между сервисами. Формат ответа при ошибке единообразен для всех сервисов (соответствует паттерну, применённому в монолите):

```
{ "code": "NOT_FOUND", "message": "User with id 42 not found" }
```

HTTP-статус	Значение
400	Некорректные входные данные (невалидный UUID, отсутствующие поля)
401	Отсутствует или недействителен X-Service-Token
404	Запрошенный ресурс не найден
422	Бизнес-правило нарушено (недопустимый переход статуса и т.п.)
500	Внутренняя ошибка сервиса

Шаги декомпозиции монолита

1. **Вынести Auth Service первым.** Наименьшее количество зависимостей; остальные сервисы будут полагаться на него для валидации токенов.
2. **Вынести User Service.** Auth Service при регистрации синхронно создаёт запись пользователя в User Service через POST /internal/users; остальные сервисы запрашивают профили через GET /internal/users/{id}.
3. **Вынести Property Service.** Зависит только от User Service (данные арендодателя).
4. **Вынести Rental Service.** Зависит от Property Service и User Service. На этом шаге внедрить паттерн Saga для атомарного изменения статуса объекта при создании аренды.
5. **Вынести Messaging Service и Review Service.** Наибольшее количество зависимостей, выносятся последними после стабилизации остальных сервисов.
6. **Внедрить API Gateway** как последний шаг для унификации входной точки и снятия BFF-логики с отдельных сервисов.
7. **Покрыть межсервисные вызовы интеграционными тестами,** убедившись в корректности контрактов /internal/... эндпоинтов перед финальным выводом из строя монолита.

Вывод

В ходе работы монолитный сервис аренды недвижимости декомпозирован на 6 микросервисов: Auth, User, Property, Rental, Messaging, Review. Каждый сервис владеет собственной базой данных (database-per-service). Спроектированы 9 внутренних REST-эндпоинтов для синхронного межсервисного взаимодействия; каждый оформлен в виде отдельной OpenAPI 3.0.3 спецификации с

описанием схем запросов, ответов и кодов ошибок. Сформированы 7 шагов поэтапной декомпозиции, обеспечивающих минимизацию рисков при переходе от монолита к микросервисной архитектуре.